

Module 02 Master Handbook: Tabular Foundations and High-Speed Data Manipulation

Cybernauts 2026 Datathon Campaign

May 2026

Abstract

The biggest separator between a beginner and a pro in a datathon is **execution speed** and **data integrity**. When you are dealing with millions of rows, using the wrong data manipulation techniques will crash your kernel or take hours to run. This handbook is a comprehensive guide to mastering the "Loop Ban," high-speed filtering, relational merging, and diagnostic data cleaning.

Contents

1 Vectorization vs. Iteration: The Loop Ban

1.1 Why Iteration is the Enemy

In standard Python, we are used to using `for` loops to process items in a list. However, in Data Science, using `for` loops to iterate over rows in a DataFrame (like using `.iterrows()`) is a **fatal mistake**.

The Performance Gap

Pandas and NumPy are built on top of **C**. When you use a vectorized operation, the calculation happens in parallel across entire columns at the hardware level. When you use a `for` loop, Python has to constantly switch context between its high-level interpreter and the low-level memory, creating a massive bottleneck.

Example: If you have 1 million rows and you want to double a column value:

- **Loop (`iterrows`):** Takes ~15-20 seconds.
- **Vectorized:** Takes ~2-5 milliseconds.

1.2 Direct Vectorized Operations

The simplest form of vectorization is performing math directly on columns. Pandas handles the alignment and parallel execution for you.

```
1 # BAD (The Loop approach)
2 for i, row in df.iterrows():
3     df.at[i, 'Total'] = row['Price'] * row['Quantity']
4
5 # GOOD (The Vectorized approach)
6 df['Total'] = df['Price'] * df['Quantity']
7
```

This applies to addition, subtraction, division, and even complex powers.

1.3 Advanced Vectorization: The Power Tools

1.3.1 Column-to-Column Logic with `np.where`

What if you need an `if-else` statement across a million rows? Use `numpy.where`. It works like the `IF` function in Excel but at lightning speed.

```
1 import numpy as np
2 # Logic: If Age > 18, set Status to 'Adult', else 'Minor'
3 df['Status'] = np.where(df['Age'] > 18, 'Adult', 'Minor')
4
```

1.3.2 Logical Mappings with `.map()`

If you have a fixed dictionary of values to replace (e.g., converting "Male/Female" to 0/1), use `.map()`.

```
1 gender_map = {'Male': 0, 'Female': 1}
2 df['Gender'] = df['Gender'].map(gender_map)
3
```

1.3.3 Row-wise Flexibility with `.apply()` and Lambda

When you have complex logic that cannot be easily expressed as a simple math operation, use `.apply()`. *Warning:* `.apply(axis=1)` is still slower than pure vectorization but significantly cleaner and faster than a raw `for` loop.

```
1 # Applying complex logic to every row
2 df['Category'] = df['Income'].apply(lambda x: 'High' if x > 100000 else '
3 Medium' if x > 50000 else 'Low')
```

1.4 Summary of Performance Hierarchy

To win the datathon, follow this priority list when writing code:

1. **Direct Vectorization:** `df['A'] + df['B']` (Fastest)
2. **NumPy Functions:** `np.where()`, `np.log()`, `np.exp()`
3. **Pandas Built-ins:** `.map()`, `.replace()`, `.isin()`
4. **Apply with Lambda:** `.apply(axis=1)` (Use only if step 1-3 are impossible)
5. **Iteration:** `.iterrows()` (**Banned**)

2 Advanced Filtering, Slicing, and Conditional Assignment

2.1 Boolean Indexing: Filtering at Scale

Boolean indexing allows you to filter rows based on a "mask" of True/False values. When you use multiple conditions, you must use **bitwise operators** instead of the standard Python **and/or**.

The Bitwise Rule

In Pandas, always use:

- **&** for **AND**
- **|** for **OR**
- **~** for **NOT**

Note: You MUST wrap each condition in parentheses () or the code will fail due to operator precedence.

```
1 # Filter rows where Age > 25 AND City is 'Dhaka'
2 mask = (df['Age'] > 25) & (df['City'] == 'Dhaka')
3 filtered_df = df[mask]
4
```

2.2 The Slicing Duo: .loc vs .iloc

Understanding the difference between these two is non-negotiable.

2.2.1 .loc[] (Label-based)

Uses column names and row index labels. *Use Case:* Accessing data when you know the "name" of the row or column.

```
1 # Syntax: df.loc[row_labels, column_labels]
2 df.loc[0:5, 'Income'] # Rows labeled 0 to 5, column 'Income'
3
```

2.2.2 .iloc[] (Integer-based)

Uses zero-based integer positions (like a standard Python list). *Use Case:* Accessing data by its "geometric position" in the matrix.

```
1 # Syntax: df.iloc[row_position, column_position]
2 df.iloc[0:5, 2] # First 5 rows, 3rd column
3
```

2.3 Conditional Assignment and the SettingWithCopyWarning

One of the most common bugs in Pandas is trying to assign values to a slice of a DataFrame. **Bad Code:** `df[df['A'] > 5]['B'] = 10` → This often fails silently or triggers a warning because you are modifying a "copy" of the data, not the original.

The Gold Standard

Always use `.loc` for conditional assignment. It guarantees you are modifying the original DataFrame.

```
df.loc[condition, 'target_column'] = new_value
```

```
1 # Set 'Risk' to High for all rows where 'Debt' > 10000
2 df.loc[df['Debt'] > 10000, 'Risk'] = 'High'
3
```

2.4 Advanced Filtering Tools

2.4.1 The .isin() Method

If you want to filter based on a list of many possible values, don't use 20 | (OR) statements. Use .isin().

```
1 top_cities = ['Dhaka', 'Chittagong', 'Sylhet']
2 df_top = df[df['City'].isin(top_cities)]
3
```

2.4.2 The .query() Method

For clean, readable code (especially with many variables), use .query(). It allows you to write filters as strings.

```
1 min_age = 18
2 df_adults = df.query("Age >= @min_age and Status == 'Active'")
3
```

3 Relational Operations: Grouping, Aggregating, and Merging

3.1 Data Aggregation: The Split-Apply-Combine Paradigm

Aggregation is the process of turning many rows into a single summary statistic. In Pandas, this follows a three-step logic:

1. **Split:** Group the data based on a categorical variable (e.g., "City").
2. **Apply:** Calculate a metric for each group (e.g., "Mean Income").
3. **Combine:** Stick the results back into a new table.

3.1.1 Basic Grouping with `.groupby()`

The most efficient way to summarize data is using the `.agg()`.

```
1 # Calculate the mean and max age per city
2 city_stats = df.groupby('City')['Age'].agg(['mean', 'max'])
3
```

3.1.2 Advanced Aggregation: Dictionary Mapping

If you need different statistics for different columns, pass a dictionary to `.agg()`.

```
1 summary = df.groupby('Category').agg({
2     'Price': 'mean',
3     'Quantity': 'sum',
4     'Customer_ID': 'count'
5 })
6
```

3.2 Merging Datasets: Database-Style Joins

When you have two tables (e.g., `df_users` and `df_transactions`), you use `.merge()` to link them.

3.2.1 The Four Join Types

The `how` parameter is the most critical part of a merge:

- **inner:** Keeps only keys that exist in **both** tables (Lossy but clean).
- **left:** Keeps all keys from the left table and adds data from the right where available (Standard for adding features).
- **right:** Keeps all keys from the right table.
- **outer:** Keeps all keys from both tables, filling missing values with NaN.

```
1 # Adding user info to transaction data
2 final_df = transactions.merge(users, on='user_id', how='left')
3
```

3.3 Structural Combinations: `.concat()`

If you have multiple files with the **exact same columns**, use `pd.concat()`.

```
1 # Stacking multiple DataFrames vertically
2 all_data = pd.concat([df_jan, df_feb, df_mar], axis=0)
3
```

4 Diagnostic Data Cleaning and Missingness

4.1 The Diagnostic Phase: Seeing the Gaps

Before cleaning, you must quantify the mess. The first step is calculating the percentage of missing values per column.

```
1 # Identifying the percentage of missing data
2 missing_pct = df.isna().sum() / len(df) * 100
3 print(missing_pct.sort_values(ascending=False))
4
```

The 40% Rule

In many datathons, if a column has more than 40-50% missing data, it may be better to drop the column entirely rather than trying to guess (impute) half the data, as you risk introducing massive bias.

4.2 Type-Based Cleaning: `select_dtypes()`

You cannot clean a "City" column the same way you clean a "Salary" column.

```
1 # Separating numeric and categorical features
2 numeric_cols = df.select_dtypes(include=['number']).columns
3 categorical_cols = df.select_dtypes(include=['object', 'category']).columns
4
```

4.3 The "Indicator" Strategy: Binary Missingness Flags

One of the most powerful tricks in competitive ML is creating an indicator column. *Why?* Because the fact that data is missing is often a signal itself.

```
1 # Create a flag before filling the data
2 df['age_was_missing'] = df['age'].isna().astype(int)
3
4 # Now fill the actual data
5 df['age'] = df['age'].fillna(df['age'].median())
6
```

4.4 Essential Cleaning Commands

4.4.1 `.fillna()` and `.replace()`

- **Imputation:** Fill missing values with a statistic.

```
1 df['Score'] = df['Score'].fillna(df['Score'].mean())
2
```

- **Handling Junk Values:** Handling strings like '?' or 'None'.

```
1 df['Workclass'] = df['Workclass'].replace('?', np.nan)
2
```

4.4.2 `.dropna()`

Use this only if missing rows are a very small percentage (< 1%).

5 Summary Practice Problems

5.1 Problem 1: Indexing Awareness

What is the difference between `df.loc[0]` and `df.iloc[0]`?

Answer: `.loc[0]` looks for the row with the **label** 0 in the index. `.iloc[0]` always returns the **absolute first row** in the current table layout.

5.2 Problem 2: Performance Choice

You have 10 million rows. You want to create a new column `Is_Expensive` that is 1 if `Price > 100` and 0 otherwise. Which method is faster: a `for` loop or `np.where`?

Answer: `np.where` is significantly faster (milliseconds vs. minutes) because it is a vectorized C-based operation.

5.3 Problem 3: Relational Logic

You have a "Students" table and a "Results" table. Some students missed the exam and are not in "Results". If you want to keep ALL students in your final table, which join type should you use?

Answer: `left` join (with Students as the left table) or an `outer` join.

5.4 Problem 4: Imputation Logic

Why should you usually use the **Median** instead of the **Mean** to fill missing values in a skewed column?

Answer: The Median is robust to outliers, whereas the Mean is heavily shifted by them.

5.5 Problem 5: Categorical Gaps

You have a categorical column "Color" with values ['Red', 'Blue', NaN]. What is a safe way to fill the missing values without guessing?

Answer: Fill with a new category like 'Unknown' or 'Missing' using `df['Color'].fillna('Unknown')`.